

雲原生 HW4 - CI/CD Pipeline

資管三 B12705041 李健安

May 16, 2026

My GitHub Repository: <https://github.com/LEE-CHIEN-AN/cicd-lab>

CI Pipeline 說明

本次作業為 cicd-lab 專案建立一條完整的 CI Pipeline，檔案路徑為 `.github/workflows/ci_b12705041.yaml`，完整內容如下：

```
1 name: CI b12705041
2
3 on:
4   push:
5     branches:
6       - '**'
7   pull_request:
8
9 permissions:
10  contents: read
11  checks: write
12
13 jobs:
14   ci:
15     runs-on: ubuntu-latest
16
17     steps:
18       - name: Checkout
19         uses: actions/checkout@v5
20
21       - name: Setup Node.js
22         uses: actions/setup-node@v5
23         with:
24           node-version: '22'
```

```

25         cache: npm
26
27     - name: Install dependencies
28       run: npm ci
29
30     - name: TypeScript typecheck
31       run: npm run typecheck
32
33     - name: Prettier check
34       run: npm run format:check
35
36     - name: Lint
37       run: npm run lint
38
39     - name: Run tests
40       run: |
41         mkdir -p reports
42         npm test -- --reporter=default --reporter=junit \
43           --outputFile.junit=reports/vitest-junit.xml
44
45     - name: Publish test results
46       uses: dorny/test-reporter@v1
47       if: always()
48       with:
49         name: Vitest Test Results
50         path: reports/vitest-junit.xml
51         reporter: java-junit

```

Pipeline 設計說明

觸發條件 (on)

Pipeline 設定在兩種情境下自動執行：

- `push` (任意 branch)：開發者每次推送程式碼，立即觸發品質檢查，確保問題在早期被發現。
- `pull_request`：開 PR 時自動觸發，在合併進 `main` 之前強制通過所有檢查，保護主線品質。

權限設定 (permissions)

- **contents: read**: 允許 Workflow 讀取 Repository 內容，`actions/checkout` 需要此權限。
- **checks: write**: 允許 `dorny/test-reporter` 將測試結果寫入 GitHub Checks 系統，使測試報告直接顯示於 Actions 頁面，而不只是可下載的 Artifact。

執行環境 (runs-on)

使用 `ubuntu-latest`，即 GitHub 提供的全新 Ubuntu 虛擬機。每次觸發都從乾淨環境開始，避免前次執行的殘留干擾，執行結束後自動刪除。

步驟說明 (steps)

Pipeline 共分為七個步驟，設計原則為「越快速的靜態檢查越優先，越耗時的動態測試越靠後」，一旦前面的步驟失敗即立即中止，避免浪費運算資源：

1. **Checkout**: 使用 `actions/checkout@v5` 將 Repository 程式碼下載至 Runner VM，是幾乎所有 Workflow 的第一步。
2. **Setup Node.js**: 使用 `actions/setup-node@v5` 安裝指定版本 Node.js (v22)。啟用 `cache: npm`，將 `node_modules` 快取，若 `package-lock.json` 未變動則直接使用快取，可省去 15-30 秒的下載時間。
3. **Install dependencies**: 執行 `npm ci`，嚴格依照 `package-lock.json` 安裝套件，確保 CI 環境與本機環境版本完全一致，避免「本機可以跑但 CI 失敗」的問題。
4. **TypeScript typecheck**: 執行 `npm run typecheck`，實際呼叫 `tsc --noEmit`，以 TypeScript 編譯器進行純型別檢查而不輸出檔案。TypeScript 型別錯誤在 Runtime 不會立即顯現，CI 在合併前提早捕捉，降低生產環境出現隱性 Bug 的風險。
5. **Prettier check**: 執行 `npm run format:check`，實際呼叫 `prettier --check .`，依照 `.prettierrc` 規定檢查所有檔案的格式 (縮排、引號、分號等)。此步驟只做檢查，不自動修正；格式不符即直接失敗，強制團隊維持統一的程式碼風格。
6. **Lint**: 執行 `npm run lint`，使用 ESLint 搭配 `@typescript-eslint` 插件進行靜態程式碼分析，偵測潛在的程式邏輯錯誤與不良實踐。`actions/setup-node` 會自動載入 ESLint 的 Problem Matcher，讓 ESLint 錯誤直接標注在 PR 的程式碼行數上。
7. **Run tests**: 執行 `npm test`，以 Vitest 框架運行所有測試。同時使用兩種 Reporter: `default` 將結果輸出至 Log，`junit` 將結果輸出為 JUnit XML 格式 (`reports/vitest-junit.xml`)，供下一步解析。
8. **Publish test results**: 使用 `dorny/test-reporter@v1` 讀取 JUnit XML，將測試結果以圖形化方式呈現於 GitHub Actions 頁面的 Checks 區塊，可直接看到每個測試案例的通過或失敗狀態。設定 `if: always()` 確保即使測試步驟失敗，報告仍會被發布，讓開發者能立即得知是哪個測試壞掉。

CI 執行結果截圖

以下為 CI Pipeline 成功執行的截圖。最新一次的 `.github/workflows/ci_b12705041.yaml` 執行所有步驟均通過，Checks 區塊顯示 Vitest 測試結果，2 個測試案例全數通過。

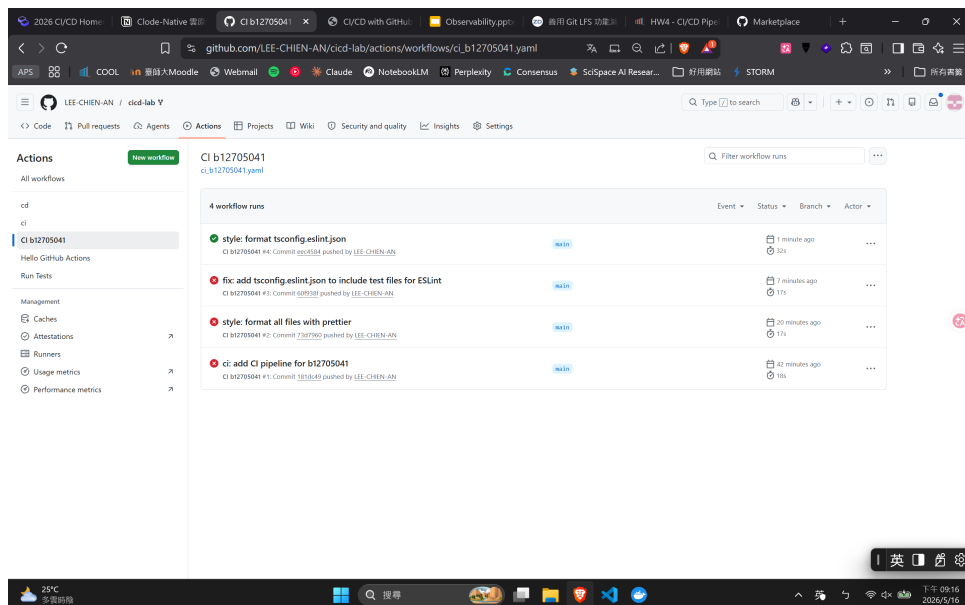


圖 1: GitHub Actions 頁面顯示最新一次的測試所有 Steps 成功通過

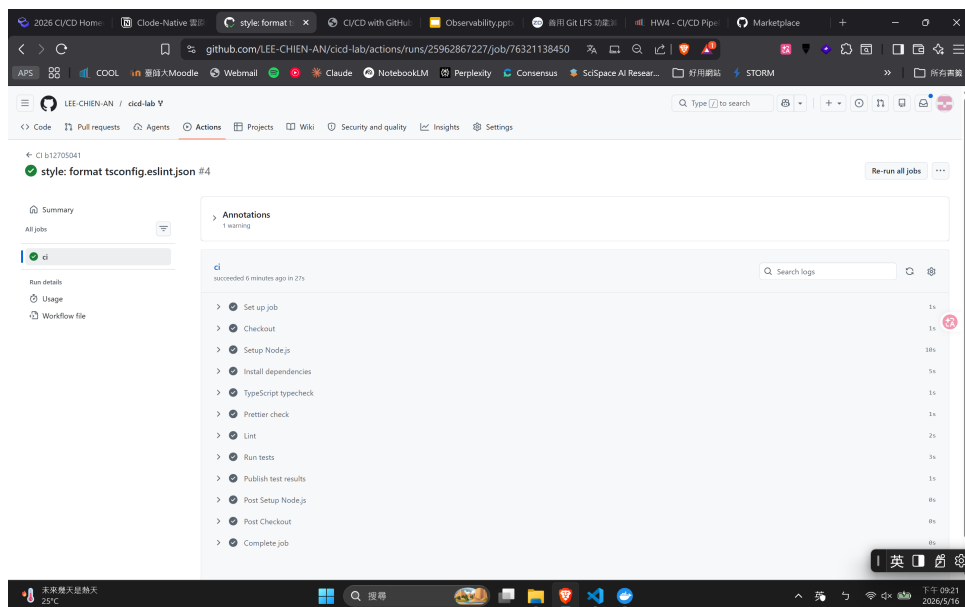


圖 2: Checks 頁面顯示 Vitest Test Results 詳細測試報告

失敗案例說明

在開發過程中共遭遇兩個導致 Pipeline 失敗的問題，以下分別說明其原因與修復方式。

案例一：Prettier 格式檢查失敗

錯誤原因

新增 Workflow 檔案後，這些 YAML 檔案的縮排與格式未符合 `.prettierrc` 的規定，CI 執行至 Prettier check 步驟時立即失敗：

```
1 [warn] .github/workflows/ci_b12705041.yaml
2 [warn] .github/workflows/01_hello.yaml
3 [warn] docker-compose.yml
4 Code style issues found in 7 files. -> exit code 1
```

連鎖反應

由於 Pipeline 在 Prettier 步驟中斷，後續的 Lint、Run tests 步驟均未執行，導致測試報告檔案（`reports/vitest-junit.xml`）從未被產生。`dorny/test-reporter` 雖因 `if: always()` 仍會執行，但找不到報告檔案，因而出現「No test report files were found」的第二個錯誤。此錯誤是 Prettier 失敗的連鎖結果，而非獨立問題。

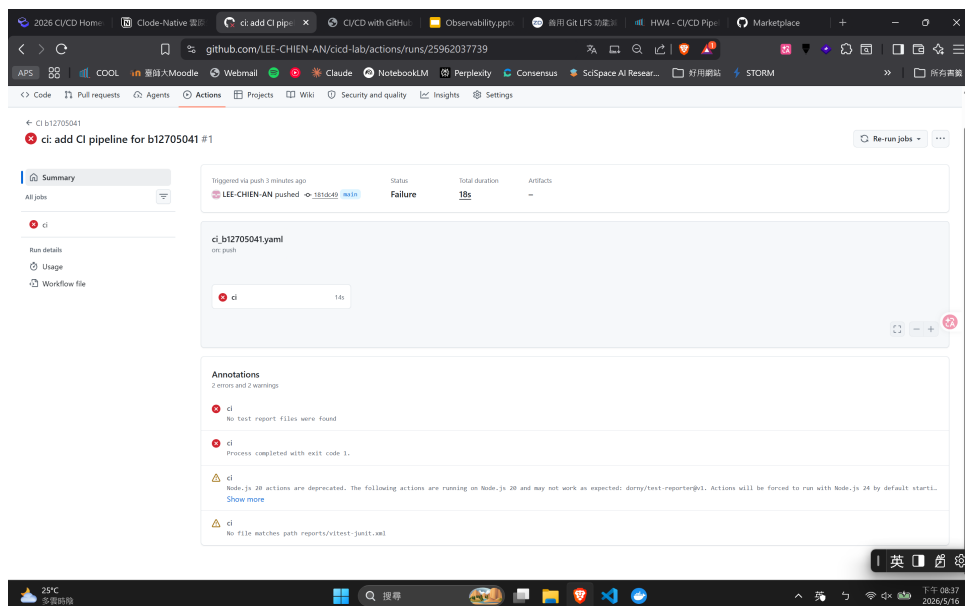


圖 3: Prettier 格式檢查失敗，Pipeline 中止

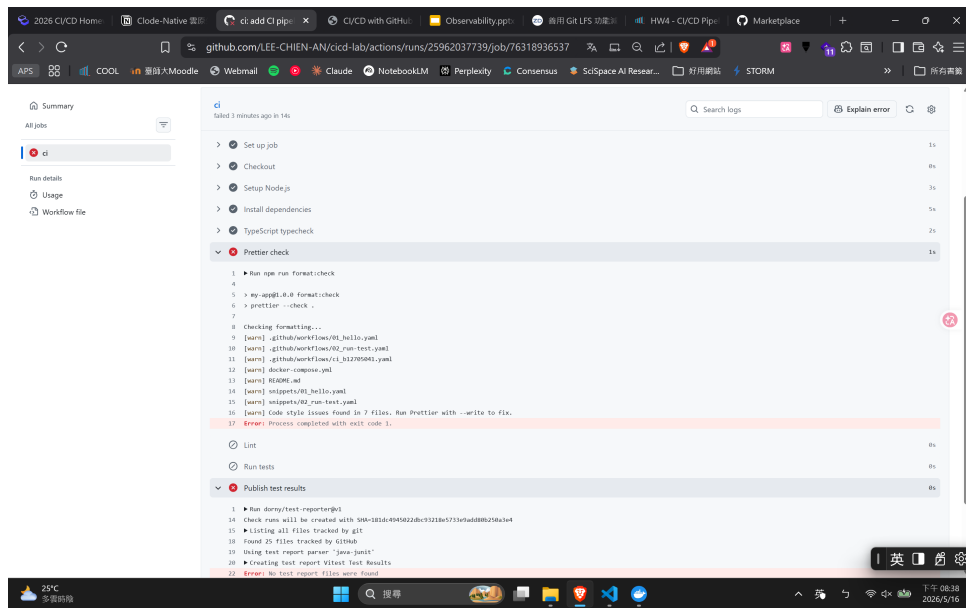


圖 4: 因測試未執行，test-reporter 找不到報告檔案

修復方式

執行 `npm run format`，讓 Prettier 自動修正所有格式問題後重新提交：

```
1 npm run format
2 git add -A
3 git commit -m "style: format all files with prettier"
4 git push origin main
```

```
PS C:\Users\ao903\Documents\slides\雲原生\cicd-lab> npm run format

> my-app@1.0.0 format
> prettier --write .

.devcontainer/devcontainer.json 84ms
.eslintrc.cjs 22ms
.github/workflows/01_hello.yaml 26ms
.github/workflows/02_run-test.yaml 12ms
.github/workflows/cd.yaml 9ms
.github/workflows/ci_b12705041.yaml 22ms
.github/workflows/ci.yaml 24ms
.prettierrc 13ms
docker-compose.yml 6ms
HW.md 101ms
package-lock.json 86ms
package.json 2ms
README.md 56ms
snippets/01_hello.yaml 5ms
snippets/02_run-test.yaml 6ms
snippets/cd.yaml 9ms
snippets/ci.yaml 10ms
src/app.ts 151ms
src/server.ts 27ms
test/app.test.ts 16ms
tsconfig.json 27ms
vitest.config.ts 14ms
```

圖 5: 執行 npm run format

案例二：ESLint 型別感知分析失敗

錯誤原因

.eslintrc.cjs 設定 `parserOptions.project: './tsconfig.json'`，要求 ESLint 使用 TypeScript 的型別資訊進行深度分析。然而 `tsconfig.json` 的 `include` 欄位只包含 `src/**/*.ts`，測試檔案 `test/app.test.ts` 與設定檔 `vitest.config.ts` 均不在其中，導致 ESLint 無法解析這些檔案並報錯：

```
1 Parsing error: "parserOptions.project" has been set for
2 @typescript-eslint/parser.
3 The file was not found in any of the provided project(s):
4 test/app.test.ts
```

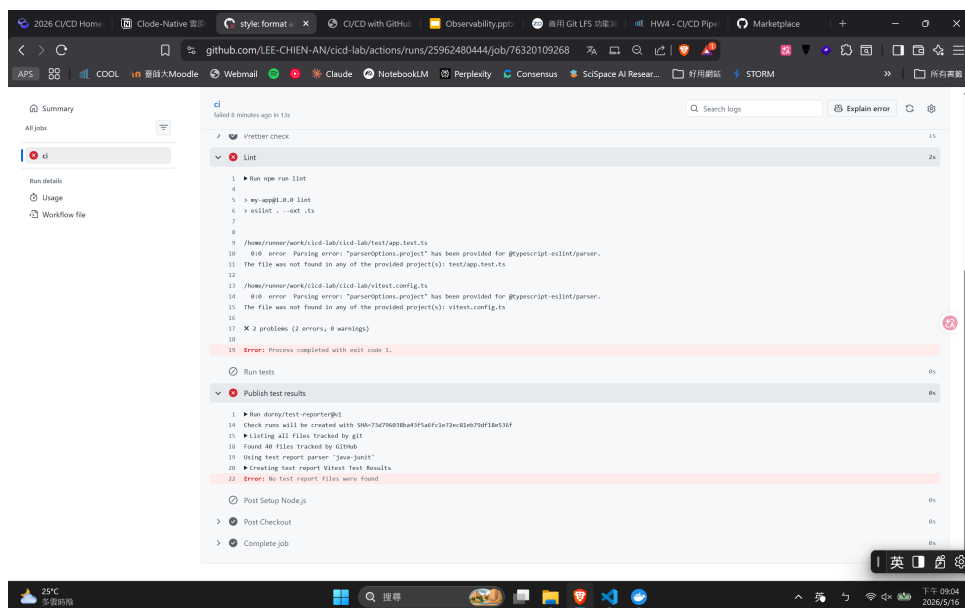


圖 6: ESLint 因 tsconfig 設定不完整而失敗

修復方式

新增 `tsconfig.eslint.json`，繼承原本的 `tsconfig.json` 設定，但額外涵蓋測試檔案與設定檔，並將 `.eslintrc.cjs` 的 `project` 指向新檔案：

```
1 // tsconfig.eslint.json
2 {
3   "extends": "./tsconfig.json",
4   "include": ["src/**/*.ts", "test/**/*.ts", "vitest.config.ts"]
5 }
```

```
1 // .eslintrc.cjs (修改前後對比)
2 // 修改前: project: './tsconfig.json'
```

```
3 // 修改後: project: './tsconfig.eslint.json'
```

此做法為 @typescript-eslint 官方推薦的標準模式，將「Build 用的 tsconfig」與「Lint 用的 tsconfig」分離，各司其職：

檔案	用途
tsconfig.json	供 npm run build 使用，只需包含 src/，測試檔案不應進入 Production Build
tsconfig.eslint.json	供 ESLint 使用，在原設定基礎上額外包含 test/ 與 vitest.config.ts

表 1: tsconfig 分離策略說明

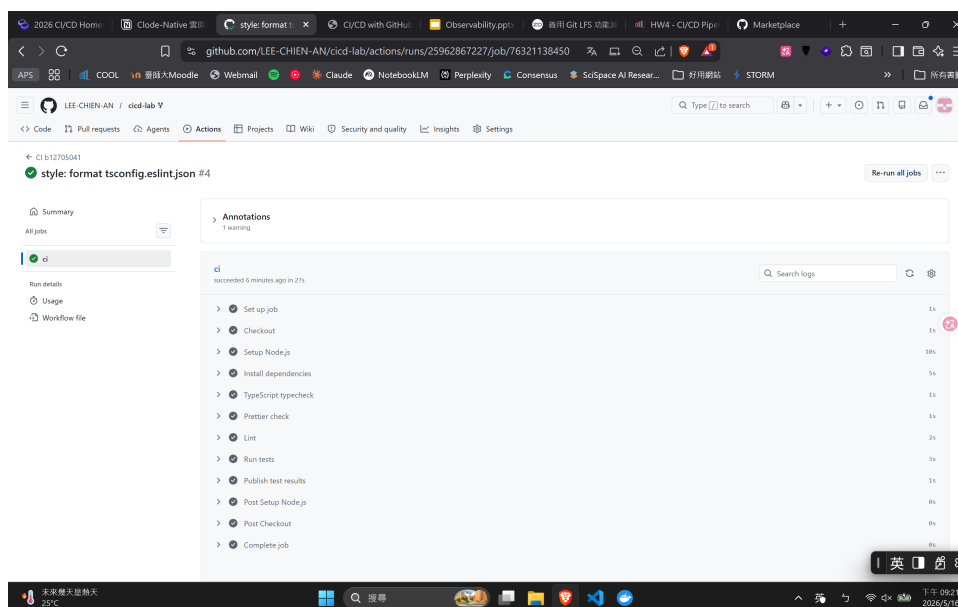


圖 7: 修復 ESLint 設定後所有步驟全數通過